

Vergleich – Kunde (Nachname, Vorname, dann Kundennr.)

Pseudocode (IHK-Stil):

```
Funktion vergleicheKunde(k1: Kunde, k2: Kunde) → Integer
    cmp = vergleicheString(k1.getNachname(), k2.getNachname())
    if cmp != 0 dann return cmp
    cmp = vergleicheString(k1.getVorname(), k2.getVorname())
    if cmp != 0 dann return cmp
    return vergleicheInt(k1.getKundenr(), k2.getKundenr())
end Funktion
```

Java-Beispiel:

```
public static int vergleicheKunde(Kunde k1, Kunde k2) {
    int cmp = k1.getNachname().compareTo(k2.getNachname());
    if (cmp != 0) return cmp;
    cmp = k1.getVorname().compareTo(k2.getVorname());
    if (cmp != 0) return cmp;
    return Integer.compare(k1.getKundenr(), k2.getKundenr());
}
```

Vergleich – Kunden (mehrstufig) – Erklärung & Beispiele

Lexikografisch: Nachname -> Vorname -> Nummer. Typische Fehler: Nur ein Feld vergleichen.

Prüfungstipp: Reihenfolge klar nennen.

k1	k2	Ergebnis
(Müller, Anna, 7)	(Müller, Bernd, 3)	-1
(Becker, Tim, 2)	(Meyer, Tom, 1)	-1
(A, A, 5)	(A, A, 9)	-1

Vergleich – Projekte (Startdatum, dann Priorität absteigend)

Pseudocode (IHK-Stil):

```
Funktion vergleicheProjekt(p1: Projekt, p2: Projekt) → Integer
    cmp = vergleicheDatum(p1.getStart(), p2.getStart())
    if cmp != 0 dann return cmp
    return - vergleicheInt(p1.getPrioritaet(), p2.getPrioritaet()) // absteigend
end Funktion
```

Java-Beispiel:

```
public static int vergleicheProjekt(Projekt p1, Projekt p2) {
    int cmp = p1.getStart().compareTo(p2.getStart()); // LocalDate
    if (cmp != 0) return cmp;
    return Integer.compare(p2.getPrioritaet(), p1.getPrioritaet()); // absteigend
}
```

Vergleich – Projekte – Erklärung & Beispiele

Primär Zeitachse (früheres Datum zuerst), sekundär höhere Priorität zuerst. Prüfungstipp:
Absteigend explizit zeigen (Parameter tauschen oder Ergebnis negieren).

p1	p2	Ergebnis
Start 01.02., Prio 3	Start 05.02., Prio 1	-1
Start 01.02., Prio 2	Start 01.02., Prio 5	+1

Vergleich – Generisch mit Schlüssel-Funktion

Pseudocode (IHK-Stil):

```
Funktion vergleicheNachKey(o1: T, o2: T, key: Funktion<T→Integer>) → Integer
    k1 = key(o1)
    k2 = key(o2)
    wenn k1 == k2 dann return 0
    sonst wenn k1 > k2 dann return +1
    sonst return -1
end Funktion
```

Java-Beispiel:

```
public static <T> int vergleicheNachKey(T o1, T o2, java.util.function.Function<T,Integer> key) {
    return Integer.compare(key.apply(o1), key.apply(o2));
}
```

Generics & Key-Funktion – Erklärung & Beispiele

Generics ermöglichen Wiederverwendung für beliebige Typen. Key-Funktion extrahiert den Sortierschlüssel (Alter, Preis, Nummer, ...). Prüfungstipp: Auf Signatur mit `<T>` und `Function<T, Integer>` hinweisen.

Objekt	key(..)	Vergleich
Person(Alter=30) vs 40	p->p.getAlter()	30<40 → -1
Artikel(Preis=9) vs 12	a->a.getPreis()	9<12 → -1

Vergleich – Mehrstufig generisch (Comparator-Kette)

Pseudocode (IHK-Stil):

```
Funktion vergleicheMehrstufig(o1: T, o2: T, k1: Funktion<T→String>, k2: Fu
    cmp = vergleicheString(k1(o1), k1(o2))
    wenn cmp != 0 dann return cmp
    return vergleicheString(k2(o1), k2(o2))
end Funktion
```

Java-Beispiel:

```
public static <T> int vergleicheMehrstufig(T o1, T o2,
    java.util.function.Function<T,String> k1,
    java.util.function.Function<T,String> k2) {
    int cmp = k1.apply(o1).compareTo(k1.apply(o2));
    if (cmp != 0) return cmp;
    return k2.apply(o1).compareTo(k2.apply(o2));
}
```

Mehrstufig generisch – Erklärung & Beispiele

Comparator-Kette: erst Schlüssel1, dann Schlüssel2. In Java:

Comparator.comparing(k1).thenComparing(k2). Prüfungstipp: Bei Strings immer compareTo verwenden.

Objekt	k1	k2	Ergebnis
(Müller, Anna) vs (Müller, Bernd)	Nachname	Vorname	Anna<Bernd → -1
(Becker, Tim) vs (Meyer, Tom)	Nachname	Vorname	Becker<Meyer → -1